Backend Task Brief

ChaosFlow - Pipeline Engine and REST API (Python / FastAPI) - 1 week

1. Overview

Build the backend of ChaosFlow, a pipeline platform where users define data pipelines (Source, Transform, Sink) as configuration, run them, and monitor them in real time. The backend has two modules: the **Pipeline Engine** and the **REST API**. Estimated effort: **1 week**.
Stack: Python 3.11+, FastAPI, SQLAlchemy 2.x, Alembic, PostgreSQL, Pydantic v2, pytest.

2. Shared Contract: Pipeline Config Schema (Day 1)

The schema below is the single source of truth shared with the Frontend team. Validate it with Pydantic, publish a JSON Schema file, and freeze it by end of Day 1. Any change after that must be announced to the team.

```
{
  "pipeline": {
    "id": "uuid",
    "name": "string",
    "version": 1,
    "schedule": null,
    "nodes": [
      { "id": "src_1", "type": "source", "plugin": "csv",
        "config": { "path": "data/orders.csv", "delimiter": "," } },
      { "id": "tr_1", "type": "transform", "plugin": "filter",
        "config": { "expression": "amount > 100" } },
      { "id": "snk_1", "type": "sink", "plugin": "postgres",
        "config": { "table": "orders_clean", "mode": "append" } }
    ],
    "edges": [
      { "from": "src_1", "to": "tr_1" },
      { "from": "tr_1", "to": "snk_1" }
    ],
    "retry": { "max_attempts": 3, "backoff_seconds": 5 }
  }
}
```

Validation rules: unique node ids; every edge references existing nodes; graph must be acyclic (DAG); at least one source and one sink; each node's `config` must validate against its plugin's own schema.

3. Module A: Pipeline Engine

Tasks

- **Plugin interfaces:** abstract classes `BaseSource`, `BaseTransform`, `BaseSink` with `validate_config()`, `open()`, `process()`, `close()`.
- **Plugin registry:** decorator-based registration and lookup by name; unknown plugin raises a clear error.
- **Sources:** CSV, JSON, REST API (with pagination), PostgreSQL (query-based).
- **Transforms:** filter, map/rename columns, cast types, aggregate (group-by), join, deduplicate.
- **Sinks:** CSV/JSON file, PostgreSQL (append / overwrite / upsert), console.
- **Executor:** topological sort of the DAG; streams records in batches; emits events (node started, batch processed, node finished, failed) through a callback.
- **Reliability:** per-node retry with exponential backoff; failures are isolated and reported with the node id and error; graceful cancellation.
- **Metrics:** records in / out, duration, and error count per node.

Deliverables

- Engine package with docstrings and type hints.
- pytest suite with at least 80% coverage (happy path, invalid config, failing node, retry, cancellation).
- CLI: `python -m engine run pipeline.json`.

4. Module B: REST API

Tasks

- **Data models:** User, Pipeline (versioned), Run, RunLog; Alembic migrations.
- **Auth:** register, login, refresh token (JWT), password hashing (argon2 or bcrypt), roles *admin* and *user*; users only see their own pipelines.
- **Pipeline CRUD** with config validation and version history.
- **Run control:** start, stop, and status of a run; runs execute in a background worker, not in the request thread.
- **Real-time:** WebSocket channel per run streaming logs, progress percentage, and records counter.
- **Plugin catalog endpoint:** lists available plugins with their config JSON Schema (used by the Frontend to render forms).
- **Cross-cutting:** consistent error format, request validation, pagination, CORS, structured logging, health check.

Endpoints

Method	Path	Purpose
POST	/auth/register, /auth/login, /auth/refresh	Authentication
GET / POST	/pipelines	List (paginated) / create
GET / PUT / DELETE	/pipelines/{id}	Read / update / delete
POST	/pipelines/{id}/runs	Start a run
POST	/runs/{id}/stop	Stop a run
GET	/runs, /runs/{id}	Run history / details
WS	/ws/runs/{id}	Live logs and progress
GET	/plugins	Plugin catalog with config schemas

Deliverables

- Working API with auto-generated OpenAPI/Swagger docs (publish the spec on Day 2 so Frontend can integrate).
- Integration tests (pytest + httpx) covering auth, CRUD, and run lifecycle.
- README with setup, environment variables, and run instructions.

5. Suggested Timeline

Day	Focus
1	Freeze config schema; project skeleton; plugin interfaces and registry
2	Sources + sinks; DB models and migrations; publish OpenAPI spec

Day	Focus
3	Transforms; executor; auth endpoints
4	Retry, metrics, cancellation; pipeline CRUD
5	Run control, background worker, WebSocket streaming
6	Tests, bug fixes, plugin catalog endpoint
7	Documentation, code review, integration with Frontend

6. Definition of Done

- All endpoints work as documented; tests pass in CI.
- A pipeline created through the API runs end to end and streams live progress.
- No hard-coded secrets; configuration through environment variables.
- Code follows PEP 8 and passes linting (ruff) and type checks (mypy).